



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

گزارش پروژه هوش مصنوعی

AI Project Report

اعضای گروه

علی فتحی

امیرعلی ملک‌لو

نام درس

هوش مصنوعی

نیم‌سال دوم ۱۴۰۴-۱۴۰۵

نام استاد درس

دکتر تن‌قطاری

۱ مقدمه و هدف پروژه

در این پروژه قصد داریم تعدادی حمله جعبه سفید به یک مدل شبکه عصبی تشخیص دسته تصویر انجام داده و بعد مدل را با روش‌های گوناگون در برابر این حملات مقاوم کنیم. همچنین در هر مرحله دقت مقاوم مدل را اندازه‌گیری کرده و در نهایت آن‌ها را با یکدیگر مقایسه می‌کنیم.

۲ توضیح فایل‌های پروژه

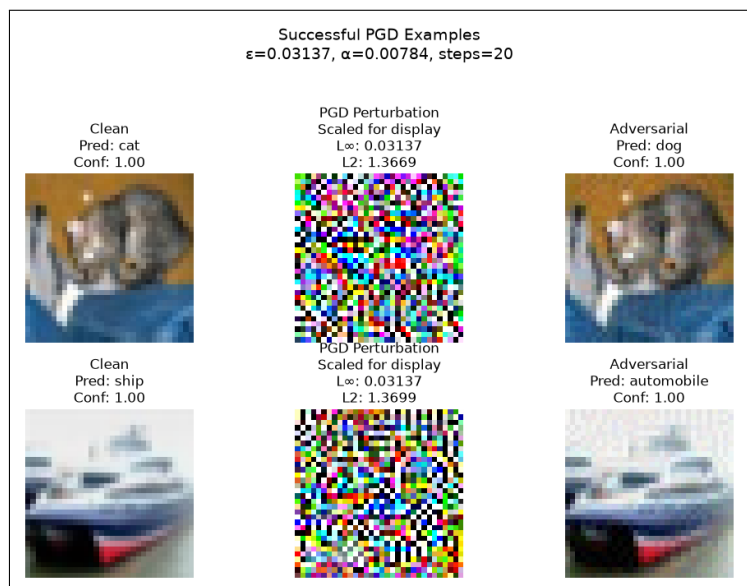
این پروژه شامل ۳ نوت‌بوک می‌باشد. در نوت‌بوک 01-adversarial_attacks_and_defenses تمامی حملات و دفاع‌ها بدون استفاده از کتابخانه مخصوص پیاده‌سازی شده و مدل‌های آموزش دیده در پوشه checkpoints ذخیره می‌شوند تا در نوت‌بوک‌های بعدی مورد استفاده قرار گیرند. در نوت‌بوک 02-adversarial_attacks_with_torchattack حملات FGSM و PGD با استفاده از کتابخانه torchattacks پیاده‌سازی شده و روی مدل‌های ذخیره شده اجرا می‌شوند. هدف از نوت‌بوک شماره ۲ این است که دقت مقاوم مدل در برابر حملات پیاده‌سازی شده با کتابخانه را با دقت مقاوم آن در برابر حملات پیاده‌سازی شده در نوت‌بوک شماره ۱ مقایسه کنیم تا از درستی پیاده‌سازی نوت‌بوک اول اطمینان حاصل شود. در نوت‌بوک 03-adversarial_training_against_C&W_attack تلاش می‌کنیم دقت مقاوم مدلی که به صورت خصمانه آموزش دیده است را در برابر حمله C&W بیشینه کنیم. به این منظور از روش‌هایی که در مقالات اشاره شده در این گزارش پیشنهاد شده‌اند استفاده می‌کنیم.

۳ مثال‌هایی از حملات

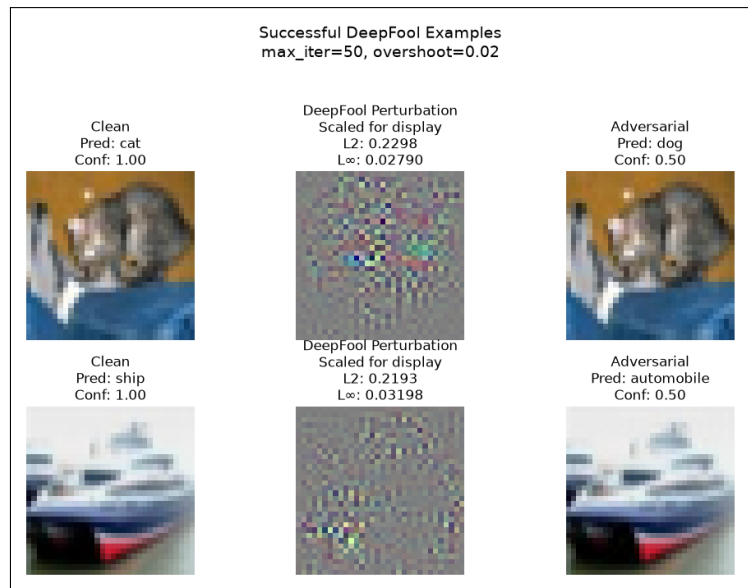
در این بخش چند مثال از نتیجه حملات انجام شده را مشاهده می‌کنید. در هر مثال، از چپ به راست ابتدا تصویر اولیه (پیش از حمله) قرار گرفته و در بالای آن دسته پیش‌بینی شده توسط مدل که دسته صحیح می‌باشد نوشته شده است، سپس نویز اعمال شده روی تصویر و در نهایت همان تصویر پس از اعمال حمله و دسته پیش‌بینی شده توسط مدل در بالای آن، که دسته غلط می‌باشد قرار گرفته‌اند.



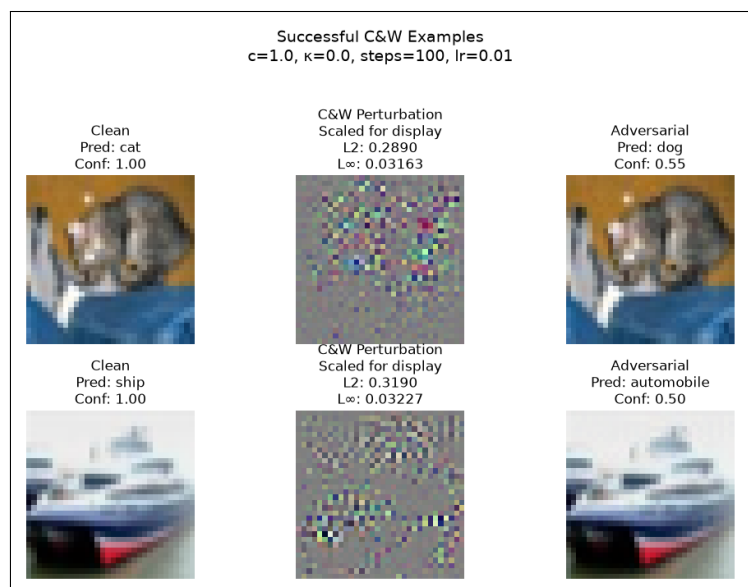
شکل ۱: مثال حمله FGSM



شکل ۲: مثال حمله PGD



شکل ۳: مثال حمله DeepFool



شکل ۴: مثال حمله C&W

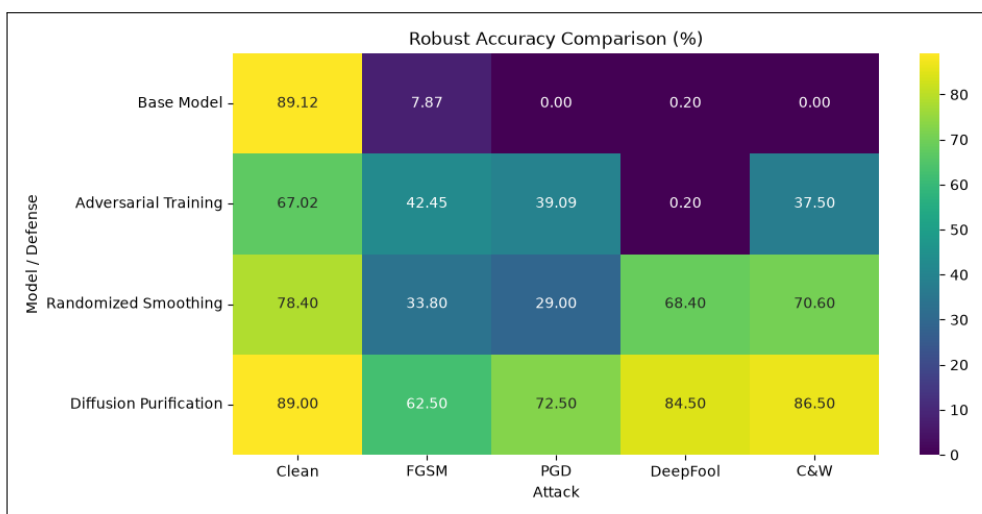


شکل ۵: مثال تمامی حملات روی یک تصویر مشترک

۴ جدول دقت مقاوم مدل‌ها در برابر حملات مختلف

C&W (L_2)	DeepFool	PGD	FGSM	Clean	دفاع / حمله
۰/۰۰	۰/۲۰	۰/۰۰	۷/۸۷	۸۹/۱۲	مدل پایه (بدون دفاع)
۳۷/۵۰	۰/۲۰	۳۹/۰۹	۴۲/۴۵	۶۷/۰۲	Adversarial Training
۷۰/۶۰	۶۸/۴۰	۲۹/۰۰	۳۳/۸۰	۷۸/۴۰	Randomized Smoothing
۸۶/۵۰	۸۴/۵۰	۷۲/۵۰	۶۲/۵۰	۸۹/۰۰	Diffusion Purification

جدول ۱: دقت مقاوم مدل‌ها (درصد) در برابر حملات مختلف



شکل ۶: Heatmap جدول دقت مقاوم

۵ پرسش‌های تحلیلی

۱. پدیده Robust Overfitting در آموزش خصمانه چیست؟ چگونه می‌توان آن را تشخیص داد و از آن جلوگیری کرد؟

در آموزش خصمانه، پدیده Robust Overfitting زمانی رخ می‌دهد که در طول فرآیند آموزش، دقت مقاوم مدل روی داده‌های خصمانه آموزشی افزایش می‌یابد ولی پس از مدتی، دقت مقاوم آن روی داده‌های تست خصمانه شروع به کاهش یافتن می‌کند. درواقع مدل به جای یادگیری الگوهای نمونه‌های خصمانه، نمونه‌ها را حفظ می‌کند و به همین خاطر دقت آن روی داده‌های آموزشی افزایش می‌یابد، در حالی که مدل توانایی تعمیم خود را از دست می‌دهد و دیگر نمی‌تواند داده‌های تست را به خوبی پیش‌بینی کند. در نتیجه مدل روی داده‌های تست خصمانه تعمیم‌پذیر نخواهد بود و در برابر حملات، مقاومت کمی از خود نشان خواهد داد. برای تشخیص آن می‌توانیم منحنی‌های accuracy و یا loss را در طول آموزش رسم کنیم. در صورتی که مشاهده کنیم دقت مقاوم مدل روی داده‌های خصمانه آموزشی در حال افزایش است ولی دقت مقاوم آن روی داده‌های تست خصمانه در حال کاهش می‌باشد، یعنی Robust Overfitting رخ داده است. یکی از راه‌های جلوگیری از این پدیده Early Stopping است. یعنی همان جایی که دقت مدل روی داده‌های تست خصمانه شروع به کاهش یافتن می‌کند، آموزش را متوقف کنیم. همچنین می‌توان از داده‌های خصمانه بیشتر، مدل ساده‌تر، و یا روش‌های Regularization مثل لایه‌های Dropout و Weight Decay استفاده کرد. در صورت امکان Data Augmentation نیز می‌تواند احتمال Overfitting را کاهش دهد.

۲. اضافه کردن Diffpure چقدر زمان استنتاج را افزایش می‌دهد؟ آیا این کندی در ازای بهبود دفاعی که ایجاد می‌کند توجیه دارد؟

برای بررسی هزینه محاسباتی DiffPure، زمان استنتاج مدل پایه و مدل مجهز به DiffPure روی ۲۰۰ نمونه اندازه‌گیری شد. نتایج به‌دست‌آمده در جدول زیر نمایش داده شده است.

روش	زمان کل (ثانیه)	زمان به ازای هر تصویر (ثانیه)
مدل پایه	۰/۰۲۶۲	۰/۰۰۰۱۳۱
DiffPure	۶۰/۰۸۷۹	۰/۳۰۰۴۳۹

جدول ۲: مقایسه زمان استنتاج مدل پایه و DiffPure

با توجه به نتایج، اضافه کردن DiffPure زمان پردازش هر تصویر را از حدود ۰/۰۰۰۱۳۱ ثانیه به ۰/۳۰۰۴۳۹ ثانیه افزایش داده است. بنابراین ضریب افزایش زمان استنتاج برابر است با:

$$\text{Slowdown Factor} = \frac{0.300439}{0.000131} \approx 2293.54$$

به عبارت دیگر، در آزمایش انجام شده، استفاده از DiffPure باعث افزایش حدود ۲۲۹۳ برابری زمان استنتاج نسبت به مدل پایه شده است.

این افزایش شدید زمان به دلیل استفاده از یک مدل Diffusion و اجرای مراحل اضافه کردن نویز و فرآیند معکوس انتشار برای بازسازی تصویر است.

از نظر دفاعی، DiffPure می‌تواند با حذف ساختار نویز خصمانه، مقاومت مدل را در برابر حملات قوی مانند PGD و C&W افزایش دهد. بنابراین در کاربردهایی که امنیت اهمیت بیشتری نسبت به سرعت دارد، این افزایش زمان ممکن است قابل قبول باشد.

با این حال، افزایش حدود ۲۳۰۰ برابری زمان استنتاج باعث می‌شود استفاده از این روش برای کاربردهای بلادرنگ یا سیستم‌هایی که محدودیت زمانی دارند، دشوار باشد. در نتیجه، مناسب بودن این روش به نوع کاربرد و نیازهای سیستم بستگی دارد.

۳. چرا Randomized Smoothing دارای گارانتی ریاضی است اما در تصاویر باکیفیت (بدون حمله) دقت کمتری نسبت به مدل اصلی دارد؟

روش Randomized Smoothing با اضافه کردن نویز گاوسی به تصویر ورودی و انجام چندین پیش‌بینی، یک مدل هموار شده ایجاد می‌کند. ایده اصلی این روش این است که به جای تصمیم‌گیری بر اساس یک تصویر، مدل چندین نسخه نویزی از همان تصویر را بررسی کرده و کلاسی که بیشترین رأی را دریافت کند به عنوان خروجی انتخاب می‌شود. این روش دارای گارانتی ریاضی است، زیرا می‌توان نشان داد اگر تغییر ایجاد شده در تصویر از یک مقدار مشخص کمتر باشد، پیش‌بینی مدل هموار شده تغییر نخواهد کرد. به همین دلیل، Randomized Smoothing می‌تواند یک محدوده تضمین شده برای مقاومت مدل در برابر برخی حملات خصمانه ارائه دهد.

با این حال، اضافه کردن نویز به تصویر باعث می‌شود ورودی مدل دیگر دقیقاً تصویر اصلی نباشد. این نویز ممکن است برخی ویژگی‌های مهم تصویر را کاهش دهد یا باعث از بین رفتن جزئیات شود. در نتیجه، حتی زمانی که تصویر فاقد حمله خصمانه است، ممکن است مدل هموار شده نسبت به مدل اصلی دقت کمتری داشته باشد.